

Short WAITS

by
Arthur Samuel

Computer Science Department
Stanford University
November 19, 1980

Short WAITS

Abstract

This is an introductory manual describing the SU-AI timesharing system that is available primarily for sponsored research in the Computer Science Department. The present manual is written for the beginner and the user interested primarily in the message handling capability as well as for the experienced computer user and programmer who either is unfamiliar with the SU-AI computer or who uses it infrequently. References are made to the available hard-copy manuals and to the extensive on-line document files where more complete information can be obtained.

The principal advantages of this system are:

1) The availability of a large repertoire of useful system features; 2) The large memory; 3) The large file storage system; 4) The ease with which one can access other computers via the ARPA net; 5) The file transfer facilities via the EFTP program and the ETHERNET; 6) The XGP printer and the large collections of fonts available for it; and 7) The fast and convenient E editor with its macro facilities.

Short WAITS

Introduction

This is an introductory manual to the SU-AI time-sharing computer intended for the beginner and as a ready reference to the more complete documentation available elsewhere. Much of the referenced material is to be found in the Monitor Command Manual that is available in hard copy as Stanford Artificial Intelligence Laboratory, Operating Note 54.6, and may be read on line by typing `READ MONCOM<CR>`. Additional documentation is available on-line in many individual files that this manual will reference.

The SU-AI computer with its WAITS operating system and the associated large, and well integrated, file system is used primarily for sponsored research.

Before you use the SU-AI computer you should make arrangements for authorization. You will be assigned some disk space to store your personal files and you will be assigned a Programmer Name which can usually be your initials or a 3-letter nickname. If you intend to work from a remote terminal, you will also need a password. If at all possible, you should begin your experience with this system using a DataDisc terminal and this manual will assume that you do.

Details as to the method of obtaining authorization may be read from any DataDisc terminal (even without being logged in) by typing `HELP LOGIN<CR>`.

1. Your Computer Terminal

There are several types of terminals currently in use on the SU-AI computer and they differ principally in regard to the number, names, and uses of the special keys that distinguish a computer terminal from a typewriter. The use of a DataDisc or DataDisc-compatible terminal will be assumed in this manual.

There is one key that is common to all currently used terminals, the `CALL` key, and it is usually located on the far upper right. This key is used to call the system

monitor. You will be interacting with the monitor when you first sit down at a terminal and whenever it prompts you by displaying a period at the beginning of a line. If the system does not print a period or if it simply fails to respond, then you may have to strike the CALL key (once or twice) to call the monitor. There are more graceful ways of getting back to the system monitor that you will learn in time, but the CALL key is always the Key of Last Resort. If the system fails to respond to the CALL key, it may be down. Either wait a few minutes and try again or check with someone at another terminal.

You should note that there is a full set of keys for the numerals and, specifically, that the letter “I” is not used for the numeral “1” and the letter “O” is not used for the numeral “0”. Confusing, isn’t? There is, of course, the usual space bar and there is a carriage return key (usually marked RETURN) that has some special functions. We will call this key the CR. Then there are a number of other keys that are not the same on different types of terminals, as will be explained below.

You may have noted the use of capitals in referring to the named keys on the keyboard. This convention will be used throughout this manual. When there might be danger of confusing the name of the key with text that is to be typed, the key’s name will be shown in angle brackets, thus, <CR>.

1.1 The DataDisc Key Board

This explanation will be easier to understand if you are at a DataDisc terminal as you read it.

In the first place, most of the keys carry two symbols and there are two different shift keys (and actually two of each kind), one kind marked SHIFT to get upper-case letters and the other kind marked TOP to type the additional characters that are shown on the tops of the keys, i.e. above the main typewriter characters.

Then there are a number of unusual non-printing keys such as the ESC, FORM, VT, BS, LINE and ALT keys. These are all very useful but you can ignore them for the moment. You may need to use some of the remaining non-printing keys even to log-in.

The CONTROL and the META keys (and there are two of each) are shift keys in that they are held down while other keys are struck. They transform the meaning of the struck keys into commands, a Monitor command if you are typing to the System Monitor, an E command if talking to the E editor, etc..

If the system (or any other program, for that matter) is typing information (and scrolling it) faster than you want to read it, type `<CONTROL><BREAK>`, that is, depress and hold the CONTROL key while you strike the BREAK key. The typeout will be interrupted and the following legend will appear at the top of your screen:

******* →HOLDING← *******

Typing `<CONTROL><CLEAR>`, that is, holding the CONTROL key and striking the CLEAR key, permits the typeout to resume.

The BREAK and CLEAR keys, when used without the CONTROL key have quite different meanings. The CLEAR key, in particular, is useful for deleting an entire line of text that you have typed in error.

1.2 Other Key Boards

A number of other types of key boards are currently being used, some, such as the earlier DataMedia key board, do not have a META key. If you are planning to use one of these, it might be well for you to seek some personal advice as to how to proceed.

2. Log-In and Log-Out

If you plan to use a DataDisc terminal, your first task will be to find a DataDisc terminal that is free, as evidenced by the following legend on the screen:

TAKE ME I'M YOURS!

There will be some other information that need not concern you for the moment. Occasionally you will find that someone's work is appearing on the screen even though no one is actually sitting at the terminal. There are ways to displace the remote user but, until you are quite familiar with what you are doing, it might be wise to find another terminal.

During busy times, and this means most afternoons, you may see something like:

Q 115 No DD channels left; type ESC CALL and wait for one

This means that the maximum permitted number of DataDisc terminals are in use. Should this be the case you can reserve a place in the queue by striking the

ESC key followed by the CALL key. The number of your terminal will be added to the list (after the letter Q). When your turn comes, your screen will go blank and you will hear a beep. You should then log in without delay.

2.1 Log-In

You log in by typing `L 1,PRG` then a carriage return, using the Programmer Name (in place of PRG) that has been assigned to you. On the DataDisc terminals the carriage return key is labelled RETURN. We will refer to this key frequently and will sometimes use the abbreviation CR or `<CR>` instead of the full name.

The comma separating the 1 from the PRG specifies that you want to receive all system messages and you want the monitor system to perform those specific tasks that you have specified in a previously established OPTION.TXT file (to be explained below). Using a slash separator instead of a comma will limit the system messages to those that are new since your last log-in. Using a period will produce a fast log-in, with no messages and with the OPTION.TXT file ignored. After you have established your initial 1 file area, you may even omit the 1, and type `L PRG` only, to get the effect of `L 1/PRG`. To summarize these and other punctuation effects:

Symbol	Meaning	Symbol	Meaning
,	All messages, uses OPTION.TXT	/	New messages, uses OPTION.TXT
.	No messages, ignores OPTION.TXT		Note: L PRG is like L 1/PRG
!	Password required, then like /	%	Sets new password, then like /

The full name for the logging-in command is LOGIN but the WAITS monitor is very tolerant and allows one to truncate commands, typing only enough letters to disambiguate the command. Also, no distinction is normally made between upper and lower case letters, but be careful, some commonly used programs are not so tolerant. It happens that two Monitor commands, namely LOGIN and LOAD, begin with LO, so LO is not an acceptable truncation while LOG and LOA are. In some cases, and the Log-In command is one of these, an initial letter is assigned, arbitrarily, to one of two or more commands that begin with the same letter. So `L 1,PRG<CR>` works.

The 1 in your log-in command says that you will be working on your project 1 and that most of the files that you will want or will create will probably be in your 1,PRG directory area. The 1 is called a PRoJect identifier (PRJ) and you may find it convenient to have several different PRJs as you accumulate files of different sorts. Any desired string of not over 3 alphanumeric characters can be

used as a PRJ, however, certain system programs expect you to have the numeral 1 as an available PRJ. The two parts, namely PRJ and PRG when taken together are frequently referred to as your PPN.

If you are required to use a password, the system will now request it. Your password will not appear on the screen as you type it, so be careful.

If everything is in order, the system will acknowledge that you are now logged in by typing something like the following:

```
Job 13      SU-AI WAITS 9.11      Assembled 09/05/80
Thursday    11-Sep-80      1138      69F  21C
```

Several messages will probably appear that will not concern you at this time. You may want to read them anyway and practice using the <CONTROL><BREAK> command to stop the typeout. Finally, the system will type a period at the beginning of a line, which is its way of telling you that it is now ready for your next command.

2.2 Automatic Log-In Services

If you have previously prepared an OPTION.TXT file, and if you use a comma or a slash (or nothing) in your log-in command, the monitor will perform certain desired actions when you log-in. An OPTION.TXT file consists of a single page with a series of commands specifying actions that are to be taken on log-in and log-out (and in response to several other commands as will be noted later). A simple file might contain the following:

```
LOGIN:  NOMAIL,WHO,BEEP,RPGSAV;
LOGOUT: RPGSAV;
```

This instructs the monitor to do four things on log-in: 1) (NOMAIL) To inform you if there is any new mail but not to ask you if you want it displayed; 2) (WHO) To display two lines of information at the top of the screen (the WHO lines); 3) (BEEP) To emit a beep when the system completes any operation taking more than 15 seconds and 4) (RPGSAV) To restore any saved TMPCOR records that were used by the system to remember COMPILE- and EDIT-type commands that you had previously used. (The name of this command (RPGSAV) is an anachronism based on the fact that an earlier program to use these records was called RPG. The present program is called SNAIL, but more on this later).

Many other log-in services are available. You might get an expert to help you with this. Later, you will want to refer to Section 5.1 in the Monitor Command Manual (available on-line by typing **READ MONCOM**<CR> to the system monitor) to learn how to establish an **OPTION.TXT** file that meets your own particular needs.

2.3 ALIAS (to switch PPNs)

You will soon feel the need for a convenient way to switch PPNs. You do this by typing **AL PRJ,PRG**<CR>, where **PRJ,PRG** is the desired default PPN to be used for all subsequent file references. If your log-in PRG is to be retained you need only type **AL PRJ**<CR>. To return to your log-in PPN you type **AL**<CR>. If you alias to another user's PPN, you may refer to his unprotected files without having to specify the PPN but you do not acquire any of the owner's special privileges and your logged-in PPN is still checked for protection purposes.

Type **HELP UFD**<CR> or **READ MONCOM**<CR> to learn how to establish other file directories under your own PRG.

2.4 Kjob (to Log Out)

To Log Out, you type **K**<CR>, that is, the letter K followed by a carriage return. This assumes that you are talking to the system monitor which, as mentioned earlier, usually prints a period at the start of a line to indicate that it is waiting for you. The system will reply by displaying a summary of your current usage that will look something like the following.

```
Job 13, [1,ALS]   Logged off TTY66      11:38  11-Sep-80
3.35 hours, console time.
0.46 minutes, cpu time.
20.25 pages average core.
Kjob
```

If you are using a telephone line, be sure that you clear the phone connection properly.

Specific commands may be appended to the **K**<CR> command in the form of so called switches. These consist of a slash followed by a single letter. For example, the command **K/S**<CR> will result in the saving of your **TMPCOR** files so that they will be automatically restored if you have the **RPSAV** option specified

on log-in in your OPTION.TXT file, as note above. Contrarywise, if you have a LOGOUT:RPGSAV; line in your OPTION.TXT file, the switch /K will countermand this order. Finally, /F will give you a fast log-out. Don't use /Z, however, unless you want all of your files Zeroed (destroyed). Fortunately the system will not believe you the first time and will ask for a confirmation.

One convenience might be mentioned—to remind yourself of something at the time of log-out, you can create a LOGOUT.MSG file containing a reminder message. The message will be typed back to you on log-out and the file will be deleted.

Remember to save any editing that you may be doing and to kill your job as soon as you are through. If you do not, the system will kill your unused job for you after a delay (as set by the system load). You may not like this, if you have failed to save your last bit of editing. In any case, it is considered anti-social to neglect to log-out as soon as possible. Also note that the system cannot hang up your phone if you are coming in over a telephone line—so be sure to do so properly.

3. Editors

Editors come in two flavors. The basic editor is the System Line Editor. You have been using this editor even as you log-in to the system. You will be using it much of the time even when you are using the E editor, since E actually relinquishes control to the System Line Editor for most of the within-a-line tasks.

The next level of editors are what might be called File Editors. There are several varieties of these, (TECO, SOS and ETV) are the best known. ETV or E is, by far, the most popular and the use of E will be assumed in this manual. You can use E to prepare and edit all of your files, including computer programs and the source files for any of the several document-generating programs that are available on the system. Actually, E contains some facilities for formatting text so one can, and some people do, prepare documents with E alone. E is well documented, both in hard-copy and in two on-line manuals.

Document-generating programs are sometimes called Document Editors. They are basically quite different from the System Line Editor and from File Editors and should, perhaps, be called Document Compilers, as they will be in this manual. Document Compilers require a well-edited input file containing text and formatting instructions and they then produce an output file that can be sent directly to a printer, with the XGP printer being the most frequently used. Some of these programs permit a limited amount of operator intervention to repair errors, but,

by far, the most common way to operate is to go back to a file editor (usually E) to edit the source file and then to reinitiate the document compiler. Three such programs will be described in a later section of this manual.

3.1 The System Line Editor

When you type commands to the system monitor and when you type text to the E editor, you will be using the System Line Editor. This Line Editor is quite special and it has many features that greatly simplify typing to the computer as compared to typing on a typewriter. You will need to know some of these features before proceeding.

You should first observe the CURSOR, an underscore mark that moves along as you type. Actually there are two underscorings when you are typing to the system monitor, one appearing under the first character of the string of characters and the second that will be one character position beyond the last character that you have typed. This second underscore will be called the CURSOR and it always shows where the next typed character will appear.

A few essential Line Editor commands are:

<BS> If you mistype a character, the BACKSPACE key (labelled BS) can be used to erase it. Striking the BS key repeatedly will continue to erase characters from the end of the line.

<CONTROL><BS> If you discover a mistyped character in an earlier portion of the line, you can move the CURSOR back to the wrongly typed character (without having to disturb the characters that follow) by first depressing the CONTROL key and holding it down while you strike the BS key to get the CURSOR back from the end of the character string, and so to immobilize the automatic erasure feature. Thereafter, you can strike the BS key as many times as necessary (either with or without holding the CONTROL key) to move the CURSOR back as far as desired. Once positioned, you need only strike the desired key to overwrite, that is, to remove the unwanted character and to replace it by the desired one.

<CONTROL><FORM> If you want to get back to the first character in the line, hold the CONTROL key and strike the FORM key. Striking the FORM key, without holding the CONTROL key, will overwrite with an unwanted character.

<CONTROL><SPACE> If you should go too far back, you can move the CURSOR forward by depressing and holding the CONTROL key while typing with the space

bar. If you strike the space bar without holding the CONTROL key, you will be overwriting the existing character at the CURSOR position with a space.

<CONTROL><TAB> Holding the CONTROL key and striking the TAB key will get you back to the end of the line. Striking the TAB key, without holding the CONTROL key, will overwrite with a TAB.

<CONTROL>D You can delete an unwanted character (and move the rest of the line of text to close the gap) by positioning the CURSOR under the unwanted character and depressing and holding the CONTROL key while you strike the letter D (for delete).

<META><char> You can insert characters (and automatically move the rest of the text along to make room), by first positioning the CURSOR at the position in the line where you want the character to appear. You then depress and hold the META key while you type the character or characters that are to be inserted.

<CR> When you have completed a line of text or a command to your satisfaction, you can terminate the line of text or command by simply striking the RETURN key without having to move the cursor to the end of the line and the entire line will be accepted for processing. If you are using the text editor E, a line marker in the form of an arrow will be advanced to the next text line.

<CONTROL><CR> This command can be used in several different situations and it produces several different responses. If you are using the Line Editor to compose a command to the System Monitor, and if you are actually in the Line Editor, then the response is identical to that for the <CR> command as just noted. If, however, you are using the Text Editor E, the response will depend upon the exact circumstances, as you will learn when you study E.

Finally, if you have just completed a monitor command by typing a <CR> and if the system fails to understand your command string, you can retrieve the string for re-editing by typing <CONTROL><CR>. Remember, this means holding the CONTROL key down while you strike the RETURN key. This is actually a Monitor command, in this case, and it only works for the last string that you have typed, so don't strike any other keys first!

The System Line Editor has many more features that you will want to know, but these few will get you started. Fortunately, most Data Disc terminals have a list of the line-editor commands displayed just above the keyboard. If you would like an up-to-date copy of this list, type `HELP DDKEY<CR>` to the system monitor. The monitor will reply:

This HELPer prints the list of Data Disc escape commands on the XGP.
Please type Y to confirm that you want this:

Terminate your Y answer with a <CR> (the Helper doesn't tell you this) and follow directions. You will be told when the XGP starts to print your request and you should go to room MJH 433 to get it without delay.

3.2 The E Editor

Your next task will be to learn how to use the E editor. You can easily do this by a session or two with the program called ETEACH. Type `HELP ETEACH<CR>` and follow directions. This will create a special file (appropriate to the terminal that you are using) on your directory area. This file will instruct you and allow you to try out what you are learning on the file itself. The file will remain in your file area, so that you may return to it at a later time, but do delete it when have learned enough to proceed. You do this by typing `DEL TEACH.DD<CR>` if you have been using a DataDisc terminal or `DEL TEACH.DM<CR>`, if using a DataMedia terminal.

You will also want to get a copy of the CSD Report STAN-CS-80-796, called ESSENTIAL E. This is more easy to read than the comprehensive on-line manual E.ALS[UP,DOC], the Manual of Last Appeal. You can refer to this on-line manual by typing `ET?<CR>` to the monitor, or while using E by holding the CONTROL key and typing the question mark symbol ? (this time without the usual <CR>, since it is a one-character command) and you can get back to the file you are editing by typing the letter H, again with the CONTROL key and without a <CR>.

4. Files and File Directories

A file is a stored collection of information, perhaps a program (either in source form or compiled), data, mail, a report or texts of any sort. Most files on the SUAI system are stored on permanently-available disk units. If you have a great deal of original data, you may find it convenient to store some of these data on magnetic tape, and if so, please refer to Appendix A 9.2 in the Monitor Command Manual and perhaps get someone to help you the first time.

The disk units are amazingly reliable and their contents are copied to back-up magnetic tapes at frequent intervals by a system program called DART. Monitor commands to run DART can be used to retrieve files that have been removed from

the disk or damaged, although, of course, the files are restored as of the last time they were saved. As an added feature, there is a command PUMPKIN that allows you to leave an order to have desired files restored for you.

4.1 File Identifiers

Files stored on the disk carry a file identifier of the form FILNAM.EXT[PRJ,PRG], where FILNAM is a name of 6 or fewer characters (usually alphanumeric). The 3- or fewer-character extension EXT is normally used to identify the file as to its type, and may be omitted for files that are not used as source files for other programs. Should you be so unfortunate as to use more than your allotted file space, some of your files may be purged and the system purger uses the extension to decide which of your files it will delete. Files with an extension of TMP are the first to go! In contrast, source files (that are properly identified) are the last to go.

Some commonly used extensions and their meanings are:

DMP	Core image file	LST	Listing output file	SAI	SAIL source file
DO	A DO file	MSG	MAIL message file	TEX	TEX source file
DOC	Document file	PAS	PASCAL source file	TMP	Temporary file
FAI	FAIL source file	PUB	PUB source file	TXT	ASCII text file
F4	Fortran source	REL	Relocatable file	XGP	File for XGP

The PPN, that is, the [PRJ,PRG] portion of the file identifier, further identifies the file and prevents it from being confused with other files that may bear the same name. This portion also specifies the file directory to which the file is assigned. For your first files, the PRJ will be the numeral 1. The PRG will be your assigned programmer name. You can read or store files bearing the PRJ,PRG that corresponds to the PRJ,PRG under which you are logged in (or aliased) without typing the brackets and the bracketed information. If you are referring to files on other directories, this information usually must be included. Because of the [PRJ,PRG], you can be as free and fanciful as you like in creating file names, but do try to make them meaningful.

Document files on [UP,DOC], [S,DOC] and on a few other public document directories follow a different convention and use the EXT to contain the Programmer Name of the person who created the file. As a convenience, documents on these directories can be read by typing READ FILNAM<CR> without an EXT and without the [PRJ,PRG]. Please note the distinction, READ expects to find the file on one of the public document directories, whereas ER and ET expect to find the file on your log-in(or alias) directory.

4.2 File Protection

In addition to the protection afforded by the use of passwords, it is possible to specify a protection code for each file directory and for each file. The code specifies the access allowed to the file for its owner (i.e. a user logged in under the file's PPN), for other authorized system users, and for guest users. The system is not totally secure, and indeed it is not intended to be since it is customary for users to share programs and data with each other. Most users do not protect their files at all except to write-protect certain critical files to prevent accidental damage. If you feel that file protection is necessary, the three-octal-digit protection code (nnn) is obtained by adding together the desired values as shown in the following table. This code is appended as a switch, /PR=nnn, to the CET command creating the file or to a RENAME command. The user may impose an additional form of protection, while reading a file, by entering it with an ER command, by using a /R switch, or by giving the E command <CONTROL>XREADONLY<CR>. See the Monitor Command Manual for method of protecting directories.

The protection code values are:

User	may not	Change PR	Read	Overwrite
Owner				100
Local user		040	020	010
Guest		004	002	001

(for files, 400 means Don't Dump, 200 means DELete protected)
(for directories, 400 means Password required for remote use only)

4.3 E Prepared Files

Files prepared by the E editor are usually divided into pages of arbitrary lengths, as dictated by the subject matter and they normally contain a directory page. This directory page is automatically kept up-to-date by E as the file is written and it helps E to locate the different pages as they are needed. This directory can also be of considerable use to the writer of the file and to others if care is taken to see that the first line on each page properly characterizes the material on the page. The HOW file is an example where this is done.

Most of the compilers and document-preparation programs on the system are written so as to recognize and ignore the directory page and to ignore the pagination.

4.4 Filehacks

FILEHACKs are simple shorthand names for certain commonly-referenced files. They usually consist of a back slash `\` followed by a name that may be truncated to only enough letters to disambiguate it from other FILEHACKs. For example, typing `ET\G<CR>` will enter the `GRIPES.TXT[2,2]` file.

Filehack names may be used with the `COPY` command and while using the `E` editor. Some useful filehacks are listed below. Some of the files are not maintained in `E` format and some of the filehacks imply `/R` (readonly) mode, as shown in the list. To override a default `/R` mode, you can explicitly specify `/-R` after the filehack.

FILEHACK	Abbr	/R?	File
<code>\BBOARD</code>	<code>\B</code>	No	The system bulletin board file
<code>\CSD</code>	<code>\C</code>	Yes	CSD bulletin board file
<code>\DIGEST</code>	<code>\DI</code>	Yes	The current AP news digest
<code>\DOWN</code>	<code>\DO</code>	Yes	The system downtime forecast file
<code>\FORWARD</code>	<code>\F</code>	Yes	The forwarding file for the MAIL system
<code>\GRIPES</code>	<code>\G</code>	No	The system gripe file
<code>\NEWS</code>	<code>\NE</code>	Yes	The current NYT news summary
<code>\NOTICE</code>	<code>\NO</code>	Yes	The system message file
<code>\NS</code>	<code>\NS</code>	Yes	Your current NS notifications file
<code>\MAIL</code>	<code>\M</code>	No	Your current mail file
<code>\PLAN</code>	<code>\P</code>	Yes	Your current plan file

The `E` editor supports two other useful filehacks, the use of the question mark `?` to refer to the on-line `E` manual and the use of the partial symbol `∂` to refer to your own mail file. Typing `ET∂<CR>` is a good way to start the day.

4.5 To DELeTe, RENaMe or COPy Files

Files may be deleted by typing `DEL FILNAM.EXT[PRJ,PRG]<CR>`. If several files are to be deleted, their names may separated by commas and typed in one command. The `[PRJ,PRG]`s may be omitted if the files are on your aliased PPN. You may prefer to use the `DIRED` program (see below) if you have several files to delete.

Files are renamed by typing `REN newname +oldname<CR>`, where the names must include extensions (if used) and PPNs if different from your alias PPN. `RENAME` may be used to change the protection by typing `REN oldname/PR = nnn<CR>` where `nnn` is the desired protection (see table under Protection above). Renaming

has the advantage over copying and deleting the original file in that only the directory references are altered and no actual copying of the file is needed.

Files are copied (without destroying the original file) by the COPY command. As always, the [PRJ,PRG] portion of the name may be omitted for files on, or to, your log-in (or aliased) area. An example will illustrate this and several other points:

```
COP XXXYYY.TMP ←XXX.ALS [UP,DOC] (2:*),YYY.TMP<CR>(3,6:9)<CR>
```

There are no such files, but if there were, a new file, containing all but the first page of file XXX with pages 3, 6, 7, 8 and 9 from file YYY (also from [UP,DOC]) appended, would appear on your logged-in (or aliased) area. Note that the PPN is sticky, that is, once one has been specified for one source file, it need not be repeated and so both source files would be from [UP,DOC]. The resulting file would, of course, not contain a directory page and you would be asked if you want one created when you try to edit the new file.

If you want to copy one or more files into your own area, as separate files, each with the same name and extension as in the source, you may omit the destination entirely and simply type:

```
COP NAM1.EX1 [PRJ,PRG],NAM2.EX2,NAM3.EX3,NAM4.EX4<CR>
```

If you list several files, watch your periods and commas, they must be right. If you try to write over a file that already exists, you will be given the option of deleting the original file or of aborting the request. If the source file or files do not exist you will be told and the <CONTROL><CR> command may be used to recall your typed string so that you may correct or modify it.

A word of caution—While PPNs are sticky, as shown, for the COPY command, different sets of rules are used in other situations.

4.6 Listing Files—SPool, XSPool, UNSPool, Qspool and DOVer

While it is possible to list files directly on the line printer and on the XGP printer, these devices are in nearly constant use and there is a serious scheduling problem. The preferred procedure is to spool the request and allow the system to schedule the actual listing on the basis of the time of the request and the projected size of the listing. Spooling has the added advantage to the user of not tying up the user's job while the listing is actually being done. Large jobs may also be scheduled for delayed spooling to a time, say at 4 A.M. (/HOLD=400) when the printer will be in less demand. The listings are normally preceded by a title page

for easy identification. Delayed listings of XGP listings are not recommended at present because of the poor stacking capability of the XGP and inadequate storage facilities, but do try to schedule large listings during off hours if at all possible.

Files are spooled for the line printer by typing `SP FILNAM.EXT[PRJ,PRG]<CR>`, and for the XGP by typing `XSP FILNAM.EXT[PRJ,PRG]<CR>`. If the PPN is not specified, your log-in or alias PPN is assumed. If there is no EXT and the given file name does not exist without an extension then LPT or XGP, whichever is appropriate, is assumed. You may follow the file name with a list of page numbers or page ranges (enclosed in parentheses) that are to be printed, for example (2,4,6:8) will cause pages 2, 4, 6, 7, and 8 to be printed.

Several files may be spooled by one command, with the file names separated by commas. The file name may carry switches. If the switch follows the name, it applies to that file only. A switch listed ahead of a file name applies to that file name and all following names.

An asterisk may be used in place of a name, an extension, a PRJ or a PRG. This will match any term except extensions of RPG, DMP or REL. This may, of course, result in the spooling of more than one file. If you use an asterisk, it might be wise to add an /ASK switch which will cause SPOOL to list the files it has selected, one at a time, and ask for a Y confirmation before spooling each file. Alternatively, the file names and associated specifications may be read from an auxilliary file if you use the @ sign followed by the name of this auxilliary file in the spool command.

Most of the files that you may want to spool will probably be normal text files, that is files written in ACSII code, but it is possible to spool binary files with the output printed in octal by appending the switch /OCTAL or /DUMP.

A few of the more useful spooling switches are:

Switch	Action
/DElete	Delete file after printing
/NOHeading	No headings at top of each page
/NOTitle	No title page
/REPeat=n	Make n (decimal) copies (not sticky)
/OCTal	Octal listing
/ASK	Ask for confirmation
/Hold=hhmm	Hold in queue until hh hours and mm minutes
/Qspool	Report status of queue after entering request

Typing `UNSP<CR>` will cause the spooler to list the files in the queue that carry your PRG, one at a time and ask if you indeed do want them removed from the

queue. Typing `Q<CR>`, or `Q/X<CR>` if you are only interested in the XGP, will cause the present contents of the queue to be listed.

It is now possible to print files on the Dover printer. This facility is still under development so `READ DOVER<CR>` for the latest details. At the present time, typing `DOV<CR>` will call this service in a hand-holding mode and you will be told what to do.

4.7 Listing and Searching File Directories

As mentioned earlier, the PPN portion of the file identifier is used to assign the file to a directory. To see a listing of a file directory, say, of the `[UP,DOC]` directory, type `DI [UP,DOC]<CR>`. For your own log-in (or aliased) directory, the PPN may be omitted or, if you want full information, type `DI/FU<CR>`. If you have more than a screen full, remember the `<CONTROL><BREAK>` (or `<HOLD>` on the DD) command to delay scrolling.

To save a copy of your directory in a file, type `DI FILNAM +<CR>`. If you have several PRJ's and want them all listed, type `DI FILNAM +[* ,PRG]<CR>`. Don't, I repeat, DON'T use `[* ,*]` or you will fill up all of your allowed file area, and maybe all of the available disk space as well, with all of the directories on the entire system!!!

Other uses of the asterisk in the DI command might be noted. For example, if you want to see the names of all files created by BH on the `UP,DOC` directory, type `DI *.BH[UP,DOC]<CR>`. If you would like a list of all of the usual document files, type `DI Z.Z[* ,DOC]<CR>`. Since there are no files of this name, only the names of the directories will be listed.

There is a new program (`R WILD<CR>`) available with greater (wild card) search capabilities than with the DI command. For details, type `READ WILD<CR>`.

4.8 Directory Editing, DRD or R DIRED

The DIRED program allows the user to edit a directory in a manner quite analogous to the way one edits a file using E. You call DIRED by typing `R DIRED<CR>` (or simply `DRD<CR>`) to the monitor, or by typing `<CONTROL>XDIRED<CR>` to E. You will be presented with a listing of the files on your login (or aliased) PPN directory. By default, this listing will be sorted alphabetically by FILNAM and

EXT. Other sortings may be specified by your OPTION.TXT file or they may be specified at the time DIREDD is called.

You can modify DIREDD's display by using commands that are similar to the E text-editor commands, and in so doing, you will be preparing a list of actions that will be executed later, either by an explicit command or by a normal exit from the program. For example, if you delete a file listing from the display by using the <CONTROL><META>D command, the file itself will be listed for deletion. You can also rename, spool, reap, unreap, and change the protection on files, all by using this program. READ DIREDD before trying anything fancy. You can abort the actual execution of your action list, if you get into serious trouble, by using the CALL key.

4.9 Monitoring File Usage, DSKSIZ and R CKSUM

The command DSKSIZ<CR> reports your disk usage and your disk allotment, or that for a specified PRG typed as an argument. Two switches may be used, /F for a fast summary or /V (for verbose) if you want to be given the opportunity of having the information recorded in a file PURGE.DAT for later reference and, perhaps, the opportunity to write a second file PURDAT to contain a list of the files for the purger to delete.

The R CKSUM command allows you to specify a list of files that you want monitored with a record kept of all pages to which any changes have been made since the last reporting. A common use of this command is to monitor changes to \BBOARD and to keep track of changes to a file when it is being worked on by several people. For full details type READ CKSUM<CR>.

5. HELP and READ HOW to learn all about it

The HELP command and the HOW file greatly simplify the task of location WAITSS system information.

5.1 HELP

The Monitor command HELP NAME<CR>, where NAME is the name of any one of many system commands or of many user-available programs, is all that you must type to get information regarding the named command or program.

In many cases, a brief resumé will appear on your screen with enough information for you to use the command or program and with references to where more information is to be found. Alternatively, you may be told of a program that can provide the service requested or that can produce a document containing the desired information and you will be asked for a Y confirmation that you want this program to be run. Typing `HELP HELPER<CR>` will supply one-line descriptions of many of these messages and services, and typing `HELP<CR>` (without a NAME), will list them.

Finally, you may be told that no HELP file exists but that a file does exist that may contain the information that you seek. The necessary `R E;FILNAM` command will appear on your screen only awaiting the typing of a `<CR>` for confirmation that you do indeed want to see this file. Most of these files are on the `UP,DOC` and `S,DOC` file directories.

Appendix H contains a complete list of all HELP resúmes, services and `R E;FILNAM` references as of 11/12/80. Typing `READ HELP<CR>` will provide information as to how you can write HELP files for the `[3,2]` directory.

Calls on HELP have the interesting and very useful feature of not being entered into the `TMPCOR` record of files visited. This means that, if one wants some information while editing a file, one can do the following: a) exit from `E` in the usual fashion; b) get HELP information either as a resumé or from a file called by a `R E;FILNAM/R` request that HELP initiates; c) exit from this called file in the usual fashion; and finally, d) return to the file one was editing by typing `ET<CR>`. One will be back at the same location in the original file just as one would have been had one not called HELP! The internal record of other files that one can return to, by the `H` command, will have been shortened to two but otherwise will be unaltered.

5.2 READ HOW

The `HOW.ALS[UP,DOC]` file is a new (and incomplete) reference file of information regarding monitor commands and useful system programs. It is organized with the material on each topic restricted to a single page and with the first lines of each page so arranged that the file's directory page forms an easily-used table of contents. The file is read by typing `READ HOW<CR>`, (how else?). The directory page can be used as a browsing aid to find out what is available. In many cases the material in the HOW file is sufficient either to allow one to do simple things without the need for further study, or to prod one's memory as to the necessary

details.

It is hoped that many users will add to the HOW file by writing HOW-formatted pages for subjects on which they are well versed. Mail such information to ALS for proofing and final inclusion in HOW. Corrections to existing pages are also welcome.

6. System Programs

Several system programs have already been mentioned on earlier pages of this manual. There are a large number of such programs that are of great value in simplifying the user's tasks. A few of these will be described in this section.

Commands like LOGIN tell the monitor to run particular system programs in your core image, that is in core space specifically assigned to you by the system. Many of these programs are evoked by simply typing their names, usually followed by some parameters. These commands are all documented in the Monitor Command Manual (READ MONCOM<CR>) with recent additions and corrections in MONCOM.UPD. Cryptic one-line descriptions of these commands are given in Appendix C.

Other equally useful programs are called by the R command; R CKSUM<CR>, is an example. These are usually documented in separate manuals. Note that R is not an abbreviation for RUN but is a different command and it requires the program to be found on a special system directory [1,3], and in fact this PPN will be used even if you should specify another one. The RUN command is used to run user's programs as will be explained later. Many of these (R FILNAM) programs are documented by HELP and HOW but a substantial number are not. A complete list of DMP files on [1,3] is given in Appendix R.

There are a few system programs that run without occupying the user's core space and hence may be called without destroying a core image that the user may wish to retain. Some of these programs may be initiated while using the E editor by special E commands, for example, to SPOOL a page of text or to MAIL some text from a file being edited to another person. Such system programs may also be initiated by an executing user's program. Attention will be called to this feature when such commands are discussed.

Just to complicate matters, there is yet another category of commands that are called by name but that actually run the SNAIL system program. SNAIL, in turn, may select and call the correct system programs to process a user's source file

and finally may load and initiate the execution of the user's compiled program, all this without further user intervention. The distinctions between these various categories of commands is somewhat artificial as far as the user is concerned and the only thing to note is that some commands are called directly by name while certain others require the R NAME method of calling (actually, some few can be called either way).

6.1 Compiling and Running User Programs, with SNAIL

Computer programs can exist and be stored as files in at least three forms, 1) as one or more Source files, written in either an Assembly language or some Higher-Level language that can be processed by an Assembler or Compiler to produce Relocatable program modules, 2) as one or more Relocatable (REL) files that can be linked together by a Link Loader to produce an executable Core Image, and 3) as a Dump (DMP) file that can be loaded directly into core for execution. User-created programs that are no longer under development may, of course, be kept in DMP form and can then be run by the the RUN command. The SNAIL program is designed for use with user-created programs while they are still undergoing development and are subject to frequent recompilations.

Some compilers actually consist of two separate programs, an initial Compiler that produces a file written in an intermediate language for some hypothetical machine and a final Translator that translates this intermediate language into the final machine-dependent form. At the present time, the SNAIL commands, available in the SNAIL program, deal only with single-stage compilers (which may, of course, involve multipasses as long as intermediate-language files are not stored and used). It is customary to write DO files that duplicate the SNAIL commands for dealing with these two-stage compilers.

SNAIL commands are actually of three classes, those having to do with the compiling of computer programs, which will be discussed in this section, those relating to document compilers and a third and less used class that incorporate some of the features of the DO commands (see the Monitor Command Manual for these).

SNAIL commands delegate to the computer many tasks relating to: 1) the checking as to the need for a recompilation, 2) the choice of the proper assembler or compiler to produce a new Relocatable (REL) file, 3) the production of an assembly listing (if wanted), 4) the choice between loading a Dump (DMP) file or link-loading the specified (REL) files to produce a Core Image, 5) the inclusion of a suitable debugging program, 6) the production of a new DMP file, and 7) the starting of

execution either of the program proper or of the debugger. SNAIL commands are remembered and may be reinitiated by giving the command only (without file names and switches) or they may be reinitiated directly from the E editor by the <CONTROL>XGO<CR> command.

Creation (or last modification) dates are kept with the Source files, the REL files and DMP files and when a SNAIL command is given, these creation dates are compared and (unless ordered otherwise by switches) the amount of work is minimized. This is done, 1) by not recompiling a source file if there is a corresponding REL file and if this REL file carries a later creation date than does the source file, and then, 2) if loading is required, by omitting the linking operation and by loading the DMP file if one exists and if this, in turn, carries a later creation date than do the REL files.

The six compiler commands and their actions (when not modified by switches), are:

SNAIL Command	Abbrev	Produce REL file	Link Load	Produce Core Image	Include Debugger	Will Start Execution with
COMPILE	COM	may				
LOAD	LOA	may	may	will		
PREPARE	PRE	may	may	will	will	
EXECUTE	EX	may	may	will		user program
TRY	TRY	may	may	will	will	user program
DEBUG	DEB	may	may	will	will	debugger

These commands take as arguments one or more file identifiers, separated by commas. Individual programs may also be broken into two (or more) files having only one END statement at the end of the last file. This is convenient if a common program portion is to be shared by more than one program. Such segment names are separated by plus signs rather than by commas, and they compile into a single REL file carrying the name of the last segment. Alternatively, the arguments (that is, the names of the files to be used by the SNAIL command and their associated switches) may be contained in a command file and referred to by an @ sign followed by the name of the command file.

The choice of the proper compiler is made by SNAIL on the basis of the file EXTs. If standard extensions are used, SNAIL will refuse to use an incorrect compiler. The file extensions need not be explicitly stated in the command if the source files carry standard extensions. In those cases where recognized extensions are not used, the desired compilers may be specified by switches. The following extensions and compiler switches are recognized:

EXT	Switch	Processor	EXT	Switch	Processor
FAI	/FAIL	FAIL assembler	SAI	/SAIL	SAIL compiler
MAC	/MACRO	MACRO assembler	F4	/F4	FORTTRAN compiler
PAL	/PALX	PALX assembler	BLI	/BLISS	BLISS compiler
MID	/MIDAS	MIDAS assembler	AL	/AL	AL compiler
S1	/S1	FASM assembler	PAS	/PASCAL	PASCAL compiler

Switches may also be used to interfere with the automatic creation-date dependencies noted above, to specify the debugger that is to be used, to produce a DMP file, and to produce various types of listings. Some of the more important switches are:

Switch	Abbrev	Meaning
/COMPILE	/COM	Re-compile always without checking the REL dates
/CONLY	/CON	Re-compile, produce an extended listing but do not load
/CREF	/C	Request a cross-reference listing
/DDT	/D	Use DDT if a debugger is required
/LIBRARY	/LIB	Search this term as a library
/LIST	/L	Make a listing file
/LONLY	/LON	Re-compile and produce only a listing file
/MAP	/MAP	Produce a loader map of global symbols
/RAID	/RA	Use RAID if a debugger is required
/SAVE	/SAV	Make a DMP file of the core image

For a complete list see MONCOM.BH[S,DOC] Section A 1.7. Switches may also be passed to the translators and to the link loader.

6.2 The DO Command

The DO command instructs the monitor to load the line-editor buffer with the text of the file named in the command, which should contain a series of commands. The system then proceeds to execute this series of commands just as if the user had typed them directly. The command string may be quite elaborate, the only proviso being that the total length of the string must not exceed the size of the buffer (140 characters). You can even circumvent this restriction by splitting your intended string of commands at a point where the command string leaves the system at monitor command level, inserting a DO command at this point calling a second DO command string (on the same page) and having the rest of the commands in this second DO command string.

Certain necessary character conversions are required in DO files to allow things like <CONTROL>, <META> and <CR> to be typed. The <CR> and <LINE> key strokes

that may be used in writing the command string are ignored, as are the E-editor directory and page marks. These special symbols are:

Symbol	Meaning
α	Adds CONTROL bit to the next character
β	Adds META bit to the next character
$\alpha\beta$	Adds both CONTROL and META bits
$\leftrightarrow$	Translates to RETURN followed by LINE
$\downarrow$	Translates to LINE
$\neq$	Translates to ALT
λ	Translates to CALL
$\otimes$	Translates to ESC
$\otimes -$	Translates to BREAK
$ $	Terminates and separates DO strings
$\equiv$	Quotes the next character (so that an α , say, can actually appear)

Note that these symbols and those used for E macros are similar but not identical, so refresh your memory before writing either type of string.

The question mark symbol $?$ performs the special function of designating the character that immediately follows it to be a variable name to be defined by the user at the time the DO command string is executed. Suppose the character is A (i.e., ?A). At the first occurrence of this ?A in the command string, DO types out A= and waits for the user to type in a text string ending with <CR>. This string (but not the <CR>) is substituted for every occurrence of ?A in the command string and the execution proceeds.

If several DO command strings are stored on the same page or in the same file, they must be separated by the $|$ symbol. The command DO T(3)<CR> calls for the execution of the third command string in the file named T.DO.

The DO command is remembered in a TMPCOR file as if it were a COMPILE-type SNAIL command, so the SNAIL-type exit-mode commands in E (XGO<CR> for example) will also repeat the DO command. Thus, users of languages that are not recognized by SNAIL can write DO files to perform similar functions.

Many of the features mentioned above can be illustrated by a DO file, named T.DO, that was used in preparing this manual and that contains the following text:

```
ET TEXT.TEX $\leftrightarrow$  $\alpha$ Xparen { $\leftrightarrow$  $\alpha$ Xdefine  $\cap$  $\leftrightarrow$  $\alpha(\alpha)\alpha\beta\downarrow\leftrightarrow$ |
R tex $\leftrightarrow$ \input text.tex $\leftrightarrow$ |
ET A?.TEX $\leftrightarrow$  $\alpha$ Xparen { $\leftrightarrow$  $\alpha$ Xdefine  $\cap$  $\leftrightarrow$  $\alpha(\alpha)\alpha\beta\downarrow\leftrightarrow$ |
R tex $\leftrightarrow$ \input A?.tex $\leftrightarrow$ |
XSPool A?.XGP/HEAD/Q/XGP/NTN=33 $\leftrightarrow$ |
```

The first string in this file (called by the monitor command `DO T<CR>`) starts E-editing the file `TEXT.TEX`. It then defines the parenthesis-finding commands to refer to the braces `{` and `}`, respectively and it defines a macro, callable by the E command `<CONTROL>␣`, that will search for a matching pair of braces, an operation that is very useful in checking `TEX` input files.

The second command string (called by the monitor command `DO T(2)<CR>`) starts the `TEX` document compiler and gives it the required input line specifying the file `TEXT.TEX`. When an error is reported by `TEX`, one can go back to E by the `TEX e` command without disturbing the retained `TMPCOR` established by the `DO T(2)` command so that `TEX` can be re-initiated from E by typing `<CONTROL>XGO<CR>`.

The third and fourth strings are similar to the first two except that the file name is given by the `?A` convention so that the name of the source file may be typed-in at the time that the `DO` command is given. The fifth string is simply a duplicate of the `XGP` spool command that `TEX` generates (with the file name in this same `?A` form).

6.3 Who, When, Where and Finger

These four commands provide the user with information as to who is currently using the system and their current activity, `WHEN` they last logged off (even if they are currently logged in), `WHERE` they are located, and puts the `FINGER` on them (identifying them by name and physical location). These commands may be given simply by typing the capitalized portions of the command names as just used (with a `<CR>`) or they may be modified by programmer names given as arguments. Without an argument, `WHEN` refers to the interrogating user only while the other three commands provide information on all users.

`WHO` presents a dynamic display that is kept updated to show who is actually getting service with some information about the total usage. `WHO` normally exits automatically after two minutes and can be terminated earlier by typing either a `<SPACE>` or a `<CR>`. Since the material to be presented will usually more than fill the screen, it may be scrolled by typing `<FORM>` or `↑` for up and `<VT>` or `↓` for down, or it may be restricted in scope (while it is running) by typing certain single letter commands (with out `<CR>`s) of which the following are the more useful:

R	Only recently running jobs	M	My jobs only
N	Restore normal display	E	Exit, leaving info. on screen

Still other commands are available that permit the detailed specification of the PPNs or job numbers that are to be displayed, either directly or indirectly through a referenced file. For full details type READ MONCOM. WHO should not be confused with the Who Lines that can be made to appear at the top of the screen by the <ESC>W command (also automatically established at log-in by an OPTION.TXT request).

The other three commands present static (possibly scrolled) displays. WHEN requires no further explanation. WHERE, without an argument, provides a list of all jobs in numerical order and a limited amount of statistical information about each job. FINGER provides a somewhat different set of details about the jobs and identifies each user by both PPN and name and gives the physical location of the terminal being used. If the FINGER request is for specific PPNs, and the individuals are not logged in, then the times of their last log-outs are reported.

6.4 FIND, DFIND and OFIND

The FIND command is used to locate information in a file by searching for a key (or several keys, separated by commas) within the specified file. The key may be any arbitrary string of characters without regard to their surroundings. The case of letters is disregarded unless specifically restricted, as noted below, and a space between words in the key will match zero or more spaces, tabs, CRLFs and formfeeds. If the file name is omitted, the system phone directory is assumed. Actually, the FIND command and its relatives DFIND (for searching the unabridged dictionary word list), and OFIND (for referencing one's OPTION.TXT file for detailed specifications), are very useful for other types of searches as well. One can, for example, search one's mail file by specifying IN ∂ or the system NOTICE.TXT file, by specifying IN ∂^* . The time taken by the search is independent of the complexity of the key, although extremely complex keys may cause a few seconds delay to initialize the search.

The key may be specified in quite general terms, using the following symbols:

Symbol	Meaning in searched-for strings
,	A comma used as string separator
≡,	A comma is actually to appear in string
≡x	A lower case x
≡X	An upper case X
{xyz}	Any of the embraced characters
∇	Any character
⊃	Any character except a delimiter
⊃	Any character except the one that follows
	Any delimiter
∞	Any number of the character that follows
∞{xyz}	Any number of any of the embraced characters
≡	Quotes the next character (so that a ∇, say, can actually appear)

Hits are reported by typing out a portion of the text. By default, a paragraph is used for the FIND command and a line is used for the DFIND command. It is also possible to set a maximum number of lines that are to be printed on each side of the hit. The hit list may be sent to a named file by adding WRITING FILNAM to the command string, and in this case only the number of hits will be reported on your terminal.

The complete syntax of the FIND, DFIND and OFIND commands is:

```
find[ within <delim>][ surround <num>] <key> [ omit[ting][ only] <omits>] [ in <file>][
writing <file>]
```

where [] indicates optional elements, the delimiter used with WITHIN must be MSG, LINE, PAGE or PARAGRAPH, and <num> specifies the maximum number of lines to surround the key. For more information READ FIND.

The E editor has an extended command <control>XDFIND that interfaces with the DFIND program (Note that the XFIND command in E is something else entirely!). This command starts up a phantom job to do the search, and a summary of the results are sent to your terminal. The summary includes the number of hits, as well as the first, last, and (if different) shortest "hit" lines. This is fine for checking the spelling of some word but take care in specifying the key, you may get either no hits or more hits than you bargained for. Incidentally, if you are a poor speller you might be interested in the SPELL program.

6.5 MAIL (to Users and to the System), SEND and TALK

The MAIL program is used to send messages to users and sometimes to special

disk files for certain purposes. All users of the system make use of the MAIL program in some form or other. Even non-authorized users are able to use MAIL to communicate with authorized users via the system. The more important commands in this group are:

Command	Meaning
MAIL	Send a message to one or more message files and notify addressees
SEND	Send a message to the terminals of one or more designated users
GRIPE	Send a message complaining about a system problem
REMIND	Schedule a message to be sent at some later time
PLAN	Create a file describing how to find you when not logged in
EVENT	Send a message to all users about an event on a given date
BATCH	Schedule the execution of a command string at some later time
LATER	Schedule the execution of a given program at some later time
ET $\emptyset$	Edit E-formatted message file
RCV	Edit non-E-formatted message file (now seldom used)
CANCEL	Delete REMIND, BATCH, or LATER requests or queued MAIL

The TALK command is a separate system feature that enables two logged-in users to link their terminals so that everything either of them types will appear on both screens. This feature is potentially dangerous, so it is wise to reach an agreement with the other party, usually through the exchange of SEND messages, prior to establishing the link.

The MAIL command, itself, will now be described. Each user of the system is automatically assigned space in a special message file, as required, to which messages can be sent. A logged-in user, on receiving a message in this way, is notified of the message arrival (normally in the lines at the bottom of the screen that are reserved for this purpose) unless a GAG command is in effect. If not logged-in, notification is given on the next log-in. The usual formats of the MAIL and SEND commands are:

MAIL PRG *message*<CR>, and SEND PRG *message*<CR>

where PRG is a programmer name and *message* is the one-line text you want sent.

If you want to mail more than one line, simply type MAIL PRG<CR> as the command and the MAIL program will ask you to type the message, which must now be terminated by <CONTROL><META><LINE>. In fact, if you simply type MAIL<CR>, the system will prompt you with requests for the name of the recipient, the names of other persons to receive copies, the subject and finally the message itself with a reminder to end your message with the usual termination of

`<CONTROL><META><LINE>`. This is the so-called *hand-holding mode*, which is more verbose and time consuming but it may be less confusing to use.

If you start to mail a one-line message and find that you need more space, terminate the first line with `<LINE` instead of `<CR>` and you can then continue to type a multi-line message.

The normal Line-Editor editing commands may be used to edit the text as you are writing it and, in fact, you can actually delete a `<CR>` and back up into an earlier line. An alternate procedure is, however, usually superior. This procedure consists of switching to E by typing `<CONTROL><META>E` (instead of the usual `<CONTROL><META><LINE>`). This causes a special `MAIL$E.TXT` file to be created and the text that you have written to be put into this file, with the command and destination information on the first page (there is no directory page) and your text on the second page. E is entered with page 2 displayed and you can now edit the file (including the first page if necessary) with all the full power of E. This only works if you are already typing the message itself. If you are still typing the command line, the correct command to switch to E is `<ALT>`. When the file has been completed and edited to your satisfaction, type `<CONTROL>XRUN<CR>`. This exits from E, sends the message and then deletes the file. Should you exit from E without typing `<CONTROL>XRUN<CR>`, you can reenter the file with E in the normal fashion and then exit by typing `<CONTROL>XRSYS MAIL<CR>`. The message will be mailed and the file deleted as before.

The MAIL schemes so far described do not preserve a copy of your outgoing mail. You could, of course, mail a copy to yourself, but an even simpler way is available. If one has an `OUTGO.MSG` file on one's normal login directory, a copy of every outgoing mail message is automatically sent to this file. The `/-OUTGO` switch will override this provision for any specific message.

The MAIL program may be evoked while in E and either the contents of the Attach Buffer or the entire page that is currently being edited may be mailed, by using the E command `<CONTROL>XMAIL PRG<CR>`. If you want to mention the subject, you must include a switch (`/SUBJECT`) and the form is `<CONTROL>XMAIL/SUBJECT PRG text of subject<cr>`.

The SEND command is similar to MAIL except that it sends the message directly to the logged-in recipient. Such messages are usually restricted to single-lines since the recipient may not be in a position to receive a multi-line message directly. If the designated recipient is not logged in, the sender is given the option of having the message mailed.

Mail may be sent to several recipients by appending the necessary number of PRGs separated by commas, or the recipients may be specified in a distribution file that is referred to by the at sign @. If the message is to be sent to all users, the asterisk may be used. You will be asked to verify that you really mean all and that perhaps you mean all local users only. If you really mean all, use two asterisks.

The other MAIL related commands are all self-documenting and can be entered in the *hand-holding* mode by simply typing their names.

6.6 The DART program

DART is a program that saves disk files on magnetic tape and restores files from tape to disk. It is used to make periodic backups of the file disk, and it is available to users who may find it necessary to restore a disk file that has been purged or otherwise lost from the disk. For full details READ DART.

6.7 Miscellaneous System Commands

A few of the many other system commands that you will soon want to be able to use might include: the RSL command to Reserve Service Level; the UNDELETE command, with a format UNDELETE newfile←oldfile, that works only if the disk space has not yet been reassigned to another file, so try it right away; the R SRCCOM command that allows one to compare two source programs, and the corresponding, but less useful, R BINCOM for binary files; the @ command (READ ATSIGN) that can be used to produce cross-reference listings. These are all described in the MONCOM manual and by other files read by the READ command.

7. Document Generators

As mentioned earlier, Document Generators accept source files and produce output files that can be sent to an output device to produce finished documents. Three such programs, POX, PUB and T_EX, are generally used. A dialect of T_EX called MAXTEX is favored by some. Separate manuals are available for these programs, both as hard copy and as on-line manuals that may be read using E.

Document generating programs, by their very nature, are rather hard to use, at least if you want to do anything at all fancy. One way to get started is to slavishly copy some document source file as to overall format and simply replace the text

by your own text. For example, if you like the format of the present manual, simply copy `SUAL.TEX[1,ALS]` into your own area under the desired name for your new file, save page 2, as is, and replace the text gradually with your own words, and then use `MAXTEX` as described below.

7.1 POX

`Pox` is the oldest and easiest to use of the available document generators. It generates an `XGP` file for the Xerox Graphics Printer and will produce justified text using a multiplicity of different fonts. Some people still find `Pox` suitable for many of their document needs.

The principal shortcomings of `Pox` are the lack of facilities for the automatic handling of such matters as Section, Footnote and Equation numberings and the automatic preparation of a Table of Contents. Confirmed users of the `TEX` program fault `Pox` for its lack of finesse in taking care of the many subtle features that are involved in producing book-quality text.

For more information on `Pox` type `READ HOW<CR>`, then use the `E` command `<CONTROL>FPOX<CONTROL>P` to get to the right page.

7.2 PUB

`PUB` is an advanced text justifier and page formatter intended primarily for use by programmers. It can automatically number pages, sections, figures, footnotes, etc. and can print their numbers in Roman numerals as well as in digit or letter form. It can generate cross references, tables of contents, and indexes. Page layout is flexible, and allows multiple column output. Line formatting includes tabs, underlining, superscripts, subscripts, centering, and justification. Macros programmed in a `SAIL`-like string-processing language can generate text to be printed in the document. The output of the compiler is a file which can be printed on the terminal, on the line printer, or on microfilm.

A complete manual exists for this editor and can be consulted on-line by typing `READ PUB<CR>`.

7.3 TEX

The `TEX` program is the best of the available document generating programs. We can do no more than to quote from the introduction in the Computer Science Dept. Report No. STAN-CS-78-675 entitled TAU EPSILON CHI. by Donald E. Knuth.

“This is a handbook about `TEX`, a new typesetting system intended for the creation of beautiful books—and especially for books that contain a lot of mathematics. By preparing a manuscript in `TEX` format, you will be telling a computer exactly how the manuscript is to be transformed into pages whose typographic quality is comparable to that of the world’s finest printers; yet you won’t need to do much more work than would be involved if you were simply typing the manuscript on an ordinary typewriter. In fact, your total work will probably be significantly less, if you consider the time it ordinarily takes to revise a typewritten manuscript, since computer text files are so easy to change and to reprocess. (If such claims sound too good to be true, keep in mind that they were made by `TEX`’s designer, on a day when `TEX` happened to be working, so the statements may be biased; but read on anyway.)”

If you cannot get a copy of this report you might buy a copy of `TEX` and `METAFONT` by D.E.Knuth, Digital Press (1979), available at the bookstore. As a last resort you might type `READ MANUAL.TEX[TEX,DEK]` to the monitor and try to read the source file from which the TAU EPSILON CHI report was prepared.

The `TEX` program is called by typing `R TEX<CR>`. It will respond by printing an `*`, and it then expects a reply of `\input FILNAM<CR>`, where `FILNAM` is a file with the extension of `TEX` that contains text and all of the necessary instructions expected by `TEX`. The `\` before the word `input` tells `TEX` that this is the symbol used in the manuscript as an escape symbol before commands. Actually any infrequently used keyboard character could be used, but the `\` is the more common. It is used in some of the auxillary input files that you may find convenient, such as the file `BASIC.TEX[1,3]`, which will load a basic set of commands. If you want to use this `BASIC.TEX` file, the first line in your file should read: `\input basic`.

7.4 MAXTEX

A preloaded version of `TEX` with macros, called `MAXTEX`, is available for the less-than-expert hacker. Don’t tell `DEK`, but this manual was prepared using `MAXTEX`.

`MAXTEX` has information on 42 fonts (named `abc...z ABC...L<>Ø;`) and uses macros to take care of such matters as: handling of chapters, sections, etc.;

automatic table of contents (as well as plates and figures); several styles of pages, with different headings; comments, annotations, footnotes; paragraphing details (enumerate, itemize, display, indent, etc.); etc.. Otherwise, it is used just as the ordinary `TeX` in the system.

`MAXTEX` is called by typing `R MAXTEX<CR>` to the monitor. There are four files, `KERMAC.TEX`, `MATMAC.TEX`, `PAPMAC.TEX` and `CODMAC.TEX`, that take the place of `BASIC.TEX` and provide a more elaborate set of macros for special features.

Documentation is somewhat frugal at the moment, see `DOCMAC.TEX[M,JLS]` for more details. Also see the `MAXTEX` page in `HOW.ALS[UP,DOC]`.

8. Interfacing other Systems

The `SUAI` computer is linked to the `ARPA` network, a facility organized by the Advanced Research Projects Agency of the Department of Defense to connect computers at various research centers funded by `ARPA`, allowing people at one host to use the resources of another host. The device which provides the interface between our computer and the network, called an `IMP` (Interface Message Processor), can be used by user programs like any other I/O device. Two main system programs are provided for connecting to other computers by console commands: the user `TELNET` program and the File Transfer Protocol program (`FTP`). The former allows you to use your terminal as if it were a terminal of the remote host computer; the latter provides high-speed transmission of data between hosts. For more information `READ TELNET` and `READ FTP`.

A link is also in the process of being developed with the `ETHERNET`. For the current file transfer protocol `READ EFTP`. Other reports are sure to be available in the near future, so ask someone about this matter.

9. UUOs and the UUO Manual

`UUOs` are monitor calls that make use of certain machine instruction codes that would otherwise be unused or illegal. Such calls are primarily of interest to those persons who program in Assembly languages, although it behoves all programmers, particularly those who use `SAIL` (or any other language that permits machine-code insertions) to at least know of the existence of such calls and of the `UUO` manual. This (276 page) manual is available on-line by typing `READ UUO<CR>`, and in hard copy as Stanford Artificial Intelligence Laboratory Operating Note 55.5. Please do

not try to get a copy unless you really need it and do not under any circumstances try spooling it.

Some of the many matters handled by UUOs have to do with establishing input and output channels, setting up buffer rings, opening files, initializing input and output devices, establishing shared upper-memory segments so that more than one user can share a program or program segment, sending and receiving inter-job messages, the getting and setting of job information, operating the Dialer, and operating video and audio switches.

Appendices

Appendix C

The following Monitor Commands are available on SU-AI WAITS 9.11/F as assembled on 10/30/80. They may be truncations as shown by capitals. This list does not include programs that are run by an R FILNAM command.

For a similar list with longer explanations READ CMDNAM<CR>. For more information try HELP NAME<CR> and if no information is available type READ MONCOM<CR>. The Index is on page 144.

@	cross-reference list with ATSIGN	DFind	find word in dictionary
ACcess	access user file directory	DIAL	talk to comp. by phone
ADvance	advance mag tape	Directory	type file statistics
ALias	set disk PPN	DO	exec. commands from file
Assign	assign device to job	DOVer	print file on Dover
ATtach	attach job to terminal	DQavg	type disk queue average
BACKspace	back mag tape	DRd	run directory editor
BATch	schedule delayed execution	DSksiz	report disk use and quota
BOok	book mode edit with E	DTn	DM simulator TELNET
CANcel	delete reminders	DUmp	dump files to mag tape
CContinue	cont. job, term. in mon. mode	E	examine core image
CDetach	cont. job, detach terminal	EDit	edit file with SOS
CEtv	create file with E	EFInd	find string (E-format)
CFork	cont. job, detach term., new job	EFTp	transfer file over EtherNet
CKmail	see if user has new mail	ELfqfix	fix jammed ELF queue lock
CLruwp	clear upper seg, write protect	EOt	mag tape positioning
COMpile	compile program	ERead	E edit file readonly mode
CONtinue	cont. job, term. in user mode	ETv	edit file with E
COPy	copy files	EVent	list event in system messages
Core	set or type core size	EXecute	compile, load, and run prog.
CREAtE	create file with SOS	Files	type status of files in use
CREf	cross-reference list with CREF	FIND	find char. string in a file
CStart	start job, term. in mon. mode	FINGer	type name and loc. of users
DAytime	type time of day or job times	Finish	close and release device
DDt	enter DDT or RAID	FIXimlac	init. IMLAC terminal
DE	deposit into core image	FLush	clear term. buffers
Deassign	deassign device from job	FOrk	detach, make new job
DEBug	compile, load prog. start debug	FTp	ARPA net file transfer
DECide	ask computer for advice	Get	load core from file
DELeTe	delete files	GRipe	report system bug
DETach	detach job from term.	HALt	stop job (↑C)

HELLO	type name of curr. mon. version	RESEt	reset a job
HELP	explain system program	RESOurces	type available resources
HOSt	find who given net host is	REStore	restore file from tape
HOT	read AP or NYT news	RETry	try again to send queued mail
KAttach	kill job, att. another	REWind	rewind mag tape
KIll	kill another job	RSI	reserve service level
Kjob	kill this job	RUn	run program from DMP file
KLog	kill this job, Login	SAve	save core image in file
LAter	run prog. later	SD	SUPDUP to some net host
LISP	run LISP	SENd	send message to user's term.
LIST	list file on line printer	SETUwp	set upper seg. write prot.
LOAd	compile and load program	SLevel	type assigned service level
LOCate	loc. dump no. for file	SPool	request line printer listing
LogIn	make a new job	SSave	save two-seg. core image
LOGOut	kill this job	Start	start job, term. in user mode
MAIL	send mail to a user	SUPdup/SD	run SUPDUP program
MAKe	create file with TECO	SYstat	type system status
NOEdit	no edit key on terminal	TAlk	talk to another terminal
NS	read AP or NYT stories	TECo	edit file with TECO
OFind	find string per OPTION.TXT	TELnet/TN	talk to ARPAnet computer
OTN	old TELNET protocol	TESt	run SYS:TEST.DMP
PDetach	detach job, make it phantom	Time	type runtime for job or system
PJob	type job using device or term.	TList	list files on dump tape
PLAn	create plan file	TN	TELNET to some net host
PLIst	list PUMPKIN requests	TRAnsfer	copy files, delete source
POx	compile doc. with POX	TRY	compile, load with debug., run
PPpn	print PPN of job	TTy	set terminal parameters
PREpare	compile, load with debug.	TURkey	who last used DART tape?
PROcess	execute command string	TYpe	type a file
PROVe	run SYS:PROVE	UDpufd	modify UDP file directory
PTty	type term. attached to job	UFd	modify DSK file directory
PUB	compile doc. with PUB	UNDelete	try to recover deleted file
PUMpkin	ask for off-line restore	UNPRotect	reduce protection of a file
Qspool	type spooler queues	UNPUmp	undo PUMPKIN requests
R	run [1,3] program	UNReap	undo REAP requests
RCv	receive mail for user	UNSpool	delete spooler request
READ	read system doc. with E	VERify	run SYS:VERIFY
REAP	mark files for dump and del.	WHEN	see when PRG logged out
REASsign	reassign device	WHERe	type job info for PPN
REEnteR	start prog. at reenter addr.	Who	display system status
REMind	create reminder message	XGplist	list file on XGP
REName	rename file or change prot.	XPart	prepare file part for XGP spool
RER	run prog. from SYS, give it text	XSpool	request XGP listing
RERUn	run prog. from DSK, give it text	Zero	clear DTA, UDP, or DSK dir.

Appendix H

HELP NAME<CR> resués, services and R E; FILNAM references are available for:

A2E	CRDIR	ETV	IIIKEY	MAP	PDLOV	SLAC	TTYESC
ACCESS	CRE	EVENT	IIPOX	MARGIE	PFORM	SLDEC	TTYJAM
ACRONY	CREF	EXAMPL	ILISP	MARS	PIX	SLIBRY	TTYSET
ADDINF	CROSS	EXERCI	IMLAC	MBOX	PK	SLR1	TVFONT
ADS	CRYPT	EXT	IMLKEY	MENTOR	PLAN	SNAIL	TYPE
ADVENT	CRYPTO	F	IMPSTA	MERGE	PLNR	SNOBOL	TYPREL
AIM248	CSET	FAIL	INDENT	METAFO	POLL	SOB	UDP
AIMDOC	D	FASBOL	INDEX	MF	PONY	SORT	UDPUFD
AIQUAL	DART	FCOPY	INFO	MICROS	POX	SOS	UFC
AIWORD	DDFONT	FCROX	IOSAIL	MIDAS	POXOLD	SOUP	UFD
AL	DDHACK	FELT	JAMLIB	MIX	PPK	SPASM	UNDELE
ALIAS	DDKEY	FILDMP	JARGON	MLISP	PPSAV	SPEED	UNXGP
ARM	DDT	FILE	KALAH	MLISP2	PRLISP	SPELL	USEM11
ARPA	DFTP	FILES	KBDMAP	MONCOM	PROGS	SPIDER	USEMUS
ARPANE	DFTPCH	FILEX	KIMKEY	MSORT	PROVE	SPINDL	USERGD
ASSIGN	DHACKS	FIND	KJOB	MTRX	PRUNE	SPOOL	UO
ATSIGN	DIABLO	FINDAF	KL10	MUS10	PTY	SPSUB	VARIAN
AUTISM	DIALNE	FIXPNT	KRL	NCOMPL	PUB	SRCOM	VISREV
ATTACH	DIAL	FINGER	KLDIF	MUSIC	PTYJOB	SRCCHK	VERIFY
AUTOLO	DIET	FLOW	KWIC	NET	PUBMAC	SSORT	WHEN
BAIL	DIR	FMUNGE	L	NETDOC	PURGE	STICKY	WHERE
BATCH	DIRE	FNTCHR	LAKOFF	NETWRK	PURGER	SUDS	WHO
BBOARD	DISPLA	FOL	LATER	NEWESC	Q	SUPDUP	WHOLIN
BCOMPL	DISPLY	FOLMRG	LAWS	NEWIO	RADIO	SUTIP	WHOPHN
BCPL	DLNFTP	FOLPRM	LCFMAN	NEWLSP	RAID	SWR	WILD
BIBOP	DLNSFT	FONT	LEHRER	NEWPRV	RAIDUP	SWRFRM	WISE
BINCOM	DM	FREEFO	LEIBNZ	NEWS	RCV	SYMBOL	WL
BKG	DMACS	FSCALE	LETTER	NOEDIT	RECAUX	SYSTEM	XGP
BLAISE	DMKEY	FTP	LIFE	NOTICE	REDUCE	TALK	XGPDVI
BLISS	DO	FUDGE2	LIFXGP	NS	REMIN	TAPE	XGPSYG
BLOOD	DOC	GENRAL	LINGO	OPOX	RESOLV	TEACH	XGPSYN
BLOO	DOVER	GEOMED	LINK	OPTION	RETRY	TECH2	XGPTYP
BOOK	DRAW	GO	LINK10	OTLSER	REVED	TECO	XIP
BUNDLE	DRD	GPRINT	LISP	P	RIVEST	TECORD	XLATE
BUREAU	DSKSIZ	GRFX	LISP16	P2P	RLISP	TELNET	XPART
CANCEL	DTLNET	GRIPE	LIST	PACK	RSL	TEMPER	XSPOOL
CANON	DTYPE	GRUMP	LOA	PAM	RUNES	TENDMP	YUMMY
CARDS	DVI	GUEST	LOADAV	PAS	SAIL	TENGWR	YUMYUM
CCLOCK	DVIXGP	HADAM	LOGIN	PASCAL	SAVE	TERMS	Z80
CGOL	E	HAR1	LOGOUT	PASDEF	SCHEME	TEST2	ZERO
CHESS	ECL	HELIB	LRNMU1	PASDOC	SCIP	TEX	370
CKMAIL	EDFONT	HELP	LRNMUS	PASHLP	SCRIBE	TFX	6500
CKSUM	EDITOR	HELPER	LSPARC	PASINS	SD	TIMES	6800
CNVR	EFIND	HOST	MACLIS	PASMAN	SEND	TIP	68KASM
CODE	EFTP	HOW	MACLSP	PASNOT	SFA	TIPUG	6TO10
COLIST	EFUN	HOWBIG	MACROS	PC	SIMPLE	TMPCOR	8080
COMBIN	EMACS	HTSWTS	MACSYM	PCHECK	SIX12	TOSCIP	
COMPIL	ESCAPE	HUNK	MAGIC	PCP	SIXSYS	TTY	
COPY	ETEACH	HYRS	MAIL	PCREF	SL	TTYCMD	

Appendix R

The following DMP files are on the [1,3] directory.

A	COMBIN	DVIXGP	FSIM2	LOADER	PACK	RDY	SYNJOB	VMR
A2E	COMPLR	E	FTP	LOGRUN	PACKMS	RECKNU	SYNTH	VNEW
ACRONY	CONVRT	EC	FUDGE2	LPS	PAGE	REDUCE	SYSDWN	VPROVE
ACRRD	COOL	ECL	FUNC	LPSYN	PAL	RELFY	TALK	VTEST
AD	COPY	EDFONT	FUNC11	LSPFNT	PALX	REPENT	TAP	VWOLF
ADAM	CPRINT	EDIT	G	LSPMON	PAM	RESOLV	TAPE	WAIT
ADH	CRAM	EFIND	GAL	MACLSP	PARRY2	RHY	TD	WAVES
ADS	CRDIR	EFTP	GEOMED	MACRO	PAS	RLISP	TEC	WEAK
ADUDP	CRE	ELCF	GLOB	MACTO6	PAS2	RMS	TECH2	WHEN
ADVENT	CREF	ENCODE	GO	MAGGOT	PASCAL	RPG	TECO	WHERE
AFPIX	CROSS	ENORM	GOGAME	MAIL	PASOLD	RSEXEC	TELNET	WHO
AID	CRU1	ERAID	GREEN	MAINSA	PC	RSL	TEMPER	WILD
AL	CRU2	ESPERA	GRHOCK	MAP	PCHECK	RUNOFF	TENDMP	WISE
ALIGN	CRYPT	EX	GRNJOB	MAXTEX	PCP	RYDER	TENGWR	WL
AM11	CRYPTO	EXB	GRUMP	META	PCPLOT	RYGAME	TEST	WLA
AMOEBA	D	EXBFTP	HARRY	METAFO	PCREF	S	TEST2	WM
AMPSCL	DA	EXHALE	HAUNT	METER	PDRILL	SILPT	TEX	WO
ARKTEX	DAEMON	EXL	HE	MEXPR	PEDIT	SAIL	TEXDOC	WOMBAT
ARMDPY	DART	EXMRG	HEART	MF	PERUSE	SAMCMP	TFDRD	X
ATSIGN	DAZZLE	EXR	HELP	MFRPVC	PETAL	SAMPLA	TFTOPL	XAP
B	DBCK	EXS	HG	MFRSYN	PFORM	SAVE	TFXGEN	XCOMPL
BASIC	DCHES	F	HOST	MFSYN	PICTUR	SAVPP	TFXPR	XGPDVI
BCOMPL	DCSTAT	F40	HOSTAB	MICRO	PIX	SCIP	TIMES	XGPJOB
BFOL	DDFONT	FAIL	HOSTAT	MICROS	PIXUR	SCORE	TINGLE	XGPLOT
BIGPIC	DDHACK	FAILSA	HYPNO	MIDAS	PK	SCRIBE	TJ6PUB	XGPQUE
BILLBD	DDJOB	FASBOL	IC	MITFIX	PLISP	SEGFND	TJCNV	XGPSYG
BINCOM	DDT	FASM	IIIGO	MIXAL	PLNR	SEGMEN	TMPCOR	XGPSYN
BLAISE	DDTLSP	FASM2	IIPOX	MIXSCR	PLOT	SFUNC	TSNAIL	XGPTYP
BLIS10	DDUSE	FCOPY	IL	MIXSND	PLTOTF	SHIT	TSTJOB	XGPVID
BLISS	DDVID	FCROX	IMPSTA	MKVID	PLTPRE	SHUFFL	TTEX	XIP
BLOOP	DEDIT	FED	INDENT	MLISP	PLTVEC	SIMPLE	TTYSET	XLATE
BLTST	DF	FILDMP	INDEX	MLISP2	PMFW	SIXTYP	TVFONT	XLD
BRAVO	DFTP	FILE	INFO	MLISPC	POINTY	SLAC	UEDDT	XLISP
BRKSND	DIABLO	FILEX	INHALE	MMFOL	POLL	SLR1	TYPJOB	XM
BROOM	DIAL	FIND	INR	MONUSE	POX	SMPLS	U	XPART
BUNDLE	DIET	FINDAF	ITSV	MP	PPK	SNAIL	UDPUSE	XS
BUREAU	DIGTIZ	FINE	JMCWC1	MPV	PPL	SOB	UEDDT	YELLOW
BUZZ	DIRE	FINGER	JMGSYN	MS	PPSAV	SOLO	UNDOC	YELO
CAL	DIZZY	FIRWRK	JOTTO	MSORT	PR	SOS	UNPACK	YH
CALLIT	DLNFTP	FIXIM1	JUST	MTRX	PREPOX	SPASM	UNPOX	ZERO
CAM	DLNSTA	FIXIML	KAFFEE	MUS10	PRIV	SPECT	UNXGP	11TTY
CANCEL	DM	FLTAPP	KALAH	MUS11	PRN	SPELL	V	12T
CANTOR	DMCHK	FMUNGE	KLMUS	MUSIC	PROFIL	SPIDER	VAR2P	1LISP
CCLOCK	DO	FNTCHR	KRL	MYMF	PROVE	SPINDL	VARIAN	3DFLY
CHACK	DOCTOR	FOL	KWIC	NCOMPL	PRUNE	SPOOL	VARL	6TO10
CHATER	DOVER	FOLISP	L	NETWHO	PSEG	SQUEEZ	VCEN	
CHECKE	DPYHAX	FOLML	LCF	NEWLCF	PTEX	SRCCHK	VCLIP	
CHRFNT	DRAW	FOLMRG	LESCAL	NEWMUS	PTY	SRCCOM	VCWP	
CHRMAC	DRONE	FONT	LIFE	NEWS	PTYJOB	SSORT	VERCH	
CHRTST	DRW	FONTCA	LIFXGP	NLISP	PTYOLD	STEREO	VERIFY	
CINEMA	DSKLOD	FORMAT	LINK	NMUSIO	PUB	STORY	VEROLD	
CKSUM	DSKTIM	FRAID	LINS	OPASCA	PUB2	SUPDUP	VIDXGQ	
CM11	DTLNET	FREEFO	LISP	OPUMA	PUMA	SURVEY	VLISP	
CODE	DTN	FRWK40	LITCRY	ORACLE	Q	SW	VM	
COLIST	DTYPE	FSCALE	LOA	P	RAID	SWR	VM15	
COMBI	DUPINS	FSIM	LOADAV	P2P	RCV	SYNDMP	VM22	

Contents

	Page
Introduction	1
1. Your Computer Terminal	1
1.1. The DataDisc Key Board	2
1.2. Other Key Boards	3
2. Log-In and Log-Out	3
2.1. Log-In	4
2.2. Automatic Log-In Services	5
2.3. ALIAS (to switch PPNs)	6
2.4. Kjob (to Log Out)	6
3. Editors	7
3.1. The System Line Editor	8
3.2. The E Editor	10
4. Files and File Directories	10
4.1. File Identifiers	11
4.2. File Protection	12
4.3. E Prepared Files	12
4.4. Filehacks	13
4.5. To DELEte, REName or COPy Files	13
4.6. Listing Files—SPool, XSPool, UNSPool, Qspool and DOVer	14
4.7. Listing and Searching File Directories	16
4.8. Directory Editing, DRD or R DIRED	16
4.9. Monitoring File Usage, DSKSIZ and R CKSUM	17
5. HELP and READ HOW to learn all about it	17
5.1. HELP	17
5.2. READ HOW	18
6. System Programs	19
6.1. Compiling and Running User Programs, with SNAIL	20
6.2. The DO Command	22
6.3. Who, When, Where and Finger	24
6.4. FIND, DFIND and OFIND	25
6.5. MAIL (to Users and to the System), SEND and TALK	26
6.6. The DART program	29
6.7. Miscellaneous System Commands	29
7. Document Generators	29
7.1. POX	30
7.2. PUB	30
7.3. TEX	31
7.4. MAXTEX	31
8. Interfacing other Systems	32
9. UUOs and the UUO Manual	32
Appendices	34
Appendix C	34
Appendix H	36
Appendix R	37